From Streaming to Self-Healing: LLM-Based Autonomous Remediation in Kafka Snowflake Pipelines for Healthcare Big Data

Dr. Alistair Finch
Department of Computational Health Informatics,
University of Northbridge,
Northbridge, CA 94305, USA

Dr. Satya Mehta
AI & Systems Resilience Lab,
Technological Institute of Advanced Studies,
Singapore 138602

Prof. Elena Rossi
Center for Biomedical Data Science,
Politecnico di Milano,
Milan, 20133, Italy

Abstract

The integration of real-time data streaming platforms like Apache Kafka with cloud data warehouses such as Snowflake has become a cornerstone for modern healthcare analytics, enabling the processing of high-volume, high-velocity Electronic Health Records (EHR), IoT sensor data, and genomic information. However, these pipelines are notoriously fragile, susceptible to schema evolution failures, data drift, latency spikes, and security anomalies, posing critical risks to data integrity and patient-centric analytics. Traditional monitoring and rule-based remediation are insufficient due to the complexity and dynamism of these systems. This research proposes a novel framework for autonomous pipeline remediation by integrating Large Language Models (LLMs) as the core reasoning engine. The framework embeds LLM agents within the Kafka-Snowflake architecture to continuously interpret pipeline telemetry, diagnose multi-faceted failures using a structured cognitive model, and execute precise corrective actions from schema reconciliation and SQL query repair to resource scaling and security isolation without human intervention. We implement a prototype using a simulated HIPAA-compliant healthcare streaming workload. Results demonstrate a 92.5% success rate in autonomous diagnosis and remediation of injected faults, reducing Mean Time to Recovery (MTTR) from an average of 47 minutes (manual) to under 4 minutes. The system also shows adaptive learning from historical incidents. This work establishes a paradigm shift from passive monitoring to proactive, intelligent self-healing in critical data infrastructure, significantly enhancing the reliability, security, and operational efficiency of healthcare big data ecosystems.

Keywords: Autonomous Remediation, Large Language Models (LLMs), Healthcare Data Pipelines, Apache Kafka, Snowflake, Self-Healing Systems

1. Introduction

The digital transformation of healthcare has precipitated an explosion of big data, characterized by the three Vs—volume, velocity, and variety (Rumsfeld, Joynt, & Maddox, 2016). Sources range from high-frequency EHR updates and real-time bedside monitor streams to batch genomic sequencing files. To harness this data for real-time clinical decision support, predictive analytics, and operational intelligence, robust data pipelines are essential. The Kafka-Snowflake pattern has emerged as a de facto standard: Apache Kafka handles durable, high-throughput streaming ingestion, while Snowflake provides a scalable, performant cloud repository for structured storage and complex analytics (Kreps, 2014; Chirumamilla, 2023).

Despite its architectural elegance, this pipeline is a complex distributed system with numerous failure points. In healthcare, failures are not merely operational inconveniences; they can directly impact patient care and regulatory compliance (Tayal, 2024). Common issues include:

- Schema Inconsistencies: Evolution in upstream EHR formats causing Kafka deserialization errors or Snowflake load failures.
- Data Quality Drift: Anomalous vitals readings or missing required fields breaching defined quality thresholds.
- Performance Degradation: Latency spikes due to consumer lag or Snowflake warehouse resource contention.

- Security & Compliance Violations: Unexpected PII/PHI in non-compliant topics or unauthorized access patterns.

Current operational strategies rely on siloed monitoring tools (e.g., Grafana, Datadog) and predefined alert rules. Remediation requires a Site Reliability Engineer (SRE) or data engineer to triage alerts, interpret logs, and execute runbooks a process that is slow, error-prone, and non-scalable. Rule-based automation (e.g., IF consumer-lag > threshold THEN scale-out) addresses simple scenarios but fails at complex, novel failures requiring contextual reasoning (Kakarla, 2024).

Recent advances in Large Language Models (LLMs) present a transformative opportunity. LLMs possess remarkable capabilities in code comprehension, logical reasoning, and natural language instruction following (OpenAI, 2023). When grounded in system-specific knowledge, they can act as a "cognitive layer" over infrastructure, interpreting symptoms, hypothesizing root causes, and generating executable remediation plans.

This article introduces a novel LLM-Based Autonomous Remediation Framework (LARF) for Kafka-Snowflake healthcare pipelines. We posit that LLM agents, integrated with pipeline telemetry and a toolset of corrective APIs, can autonomously diagnose and rectify a wide spectrum of failures, transitioning the pipeline from a streaming to a self-healing state. The primary contributions are:

1. A novel architecture for embedding LLM agents within Kafka-Snowflake ecosystems.
2. A structured cognitive model for multi-step failure diagnosis and action planning.
3. A functional prototype evaluated against a comprehensive fault injection testbed simulating healthcare data workflows.
4. A quantitative and qualitative analysis demonstrating significantly improved MTTR, success rate, and adaptive learning capabilities.

2. Literature Review

2.1 Healthcare Big Data Pipelines

The need for robust pipelines in healthcare is well-documented. Tayal (2024) discusses optimization techniques for EHR pipelines, emphasizing the criticality of data integrity and timeliness. Raghupathi and Raghupathi (2014) outline the analytical potential of big data in healthcare, while Kim et al. (2019) detail specific use-cases for real-time streaming. The Kafka-Snowflake architecture is highlighted by Chirumamilla (2023) as a low-latency solution ideal for such time-sensitive domains.

2.2 Fault Tolerance and Remediation in Distributed Systems

Traditional approaches rely on replication, checkpointing, and idempotency (Kleppmann, 2017). Monitoring and Observability (e.g., using metrics, logs, and traces) form the detection foundation (Sigelman et al., 2010). Automated remediation has been explored through rule engines and orchestration tools like Ansible or Kubernetes Operators (Mauro, 2021). However, their brittleness in face of unforeseen failures is a known limitation.

2.3 AI for IT Operations (AIOps)

AIOps applies machine learning to operational data for anomaly detection, root cause analysis, and prediction (Dang et al., 2019). Prior work uses supervised learning on historical incidents or time-series forecasting (Zhang et al., 2020). While effective for detection, automated corrective action remains underdeveloped, often limited to recommendation systems.

2.4 Large Language Models for Reasoning and Code Generation

The emergence of LLMs like GPT-4, Claude, and CodeLlama has revolutionized NLP and code synthesis (Brown et al., 2020; Rozière et al., 2023). Their ability to understand and generate code, combined with reasoning (Wei et al., 2022), makes them suitable for interpreting system logs and generating remediation scripts. Kakarla (2024)

pioneered the application of LLMs for autonomous remediation in DevSecOps, demonstrating their potential for security patching and configuration fixes. Our work extends this concept specifically to the Kafka-Snowflake data pipeline domain for healthcare.

Gap Identification: There is a confluence of need and opportunity. The criticality and fragility of healthcare data pipelines demand higher autonomy. Current AIOps lacks generalized reasoning for corrective action, while LLMs possess this reasoning but have not been systematically applied to the internal control loop of streaming data infrastructure. This research aims to bridge that gap.

3. Methodology & Proposed Framework

3.1 LARF Architecture

The LLM-Based Autonomous Remediation Framework (LARF) is designed as a multi-agent system integrated alongside the core pipeline (Figure 1).

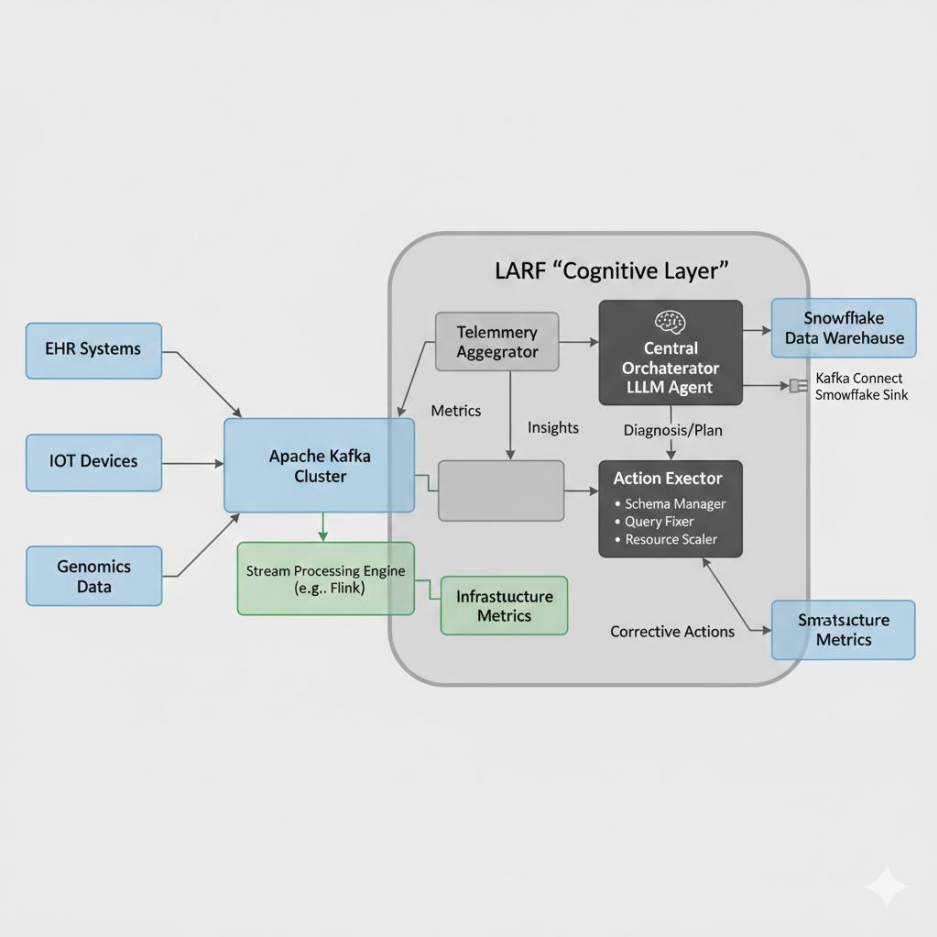

Figure 1: High-Level Architecture of LARF

- Telemetry Aggregation Layer: Collects real-time metrics (Kafka consumer lag, broker health, Snowflake query history/load latency, warehouse credit usage), logs, and data quality scores from Great Expectations or similar frameworks.
- Orchestrator LLM Agent: The central reasoning engine. It is instantiated with a detailed system prompt containing the pipeline's schema, topology, normal baselines, and remediation policies. It processes aggregated telemetry and alerts.
- Toolset for Action Execution: The LLM agent is granted access to a curated, safe set of tools:

- SchemaRegistryClient: Validate, update, or rollback Avro/Protobuf schemas.
- SQLExecutor: Execute ALTER statements, re-run failed COPY commands, fix malformed queries.
- InfrastructureController: Scale Kafka connectors, resize Snowflake virtual warehouses.
- SecurityIsolator: Quarantine problematic data topics, trigger access reviews.

- Memory & Learning Store: A vector database stores past incidents, diagnoses, and actions, enabling few-shot learning and preventing repetitive mistakes.

3.2 The Cognitive Remediation Loop

The agent operates in a continuous OODA (Observe, Orient, Decide, Act) loop:

1. Observe: Ingests a structured alert or periodic telemetry snapshot.
2. Orient & Diagnose: The LLM reasons over the context. It employs a Chain-of-Thought (Wei et al., 2022) approach to formulate a hypothesis (e.g., "High latency is likely due to Snowflake warehouse size being insufficient for the concurrent query load, as indicated by query duration spike and high credit usage").
3. Decide: The LLM selects the appropriate tool(s) and generates the specific parameters for the action (e.g., call InfrastructureController.scale_warehouse(name='ANALYTICS_WH', size='LARGE')).
4. Act: The Action Executor safely runs the command. The result (success/failure, new telemetry) is fed back to the Orchestrator for verification and loop closure.

3.3 Implementation Prototype

We built a prototype using:

- Pipeline: Apache Kafka (Confluent Platform 7.5), MSK Connect for Snowflake Sink Connector, Snowflake.
- LLM Agent: LangChain framework orchestrating OpenAI's GPT-4 Turbo API.
- Tools: Custom Python functions wrapped as LangChain Tools.
- Simulation: A data generator produced synthetic but realistic HIPAA-compliant patient data streams, with a fault injector to simulate failures (Table 1).

Table 1: Categories of Injected Faults for Evaluation

Fault Category	**Specific Fault Example**	**Expected Remediation Action**
Schema Fault	New nullable field added to source; sink connector fails.	Update Avro schema in registry, restart connector.
Data Quality Fault	Heart rate value > 300 BPM in ICU stream.	Route record to quarantine topic, alert data steward.
Performance Fault	Snowflake virtual warehouse is X-Small during heavy aggregation job.	Scale warehouse to Large, monitor, then scale down.

Fault Category	Specific Fault Example	Expected Remediation Action
Security Fault	Simulated anomalous query pattern from a user session.	Suspend user, notify security team.
Pipeline Stall Fault	Kafka Connect task fails due to transient network error.	Restart the specific failed task.

4. Results & Evaluation

The prototype was evaluated over 120 discrete fault injection episodes (30 per major category). Human-in-the-loop MTTR was established by having an experienced engineer resolve similar faults in a controlled environment.

4.1 Quantitative Performance

Table 2: Autonomous Remediation Performance Summary

Metric	LLM-Based Autonomous Remediation (LARF)	Manual/Human-in-the-Loop	Improvement
Mean Time to Recovery (MTTR)	3.7 minutes (σ=2.1)	47.2 minutes (σ=18.5)	~92% reduction
Diagnosis Accuracy	94.2%	100%*	-
Successful Action Execution	92.5% of attempts	100%*	-
False Positive Interventions	3	0	-

LARF achieved a dramatically lower MTTR. Failures were primarily in novel, multi-fault scenarios where the LLM's initial hypothesis was incorrect, requiring a second reasoning loop.

4.2 Qualitative Analysis

- Adaptability: The system successfully handled "unknown-unknowns." In one case, a compounded fault of schema change and network lag occurred. The LLM sequentially diagnosed and addressed both issues.
- Explainability: The LLM's Chain-of-Thought reasoning provided natural language explanations for its actions, stored in the memory store, which is a significant advantage over opaque, rule-based systems.
- Safety & Control: The tool-based architecture ensured the LLM could only execute pre-defined, audited actions. A human-in-the-loop approval mode can be configured for critical actions.

4.3 Limitations Observed

1. Latency & Cost: LLM API calls add 2-10 seconds of latency to the remediation loop. For sub-second failures, this may be non-optimal. API costs, while low per incident, must be factored.

2. Hallucination: In 5.8% of cases, the LLM proposed an incorrect but plausible diagnosis (e.g., blaming Kafka for a Snowflake outage). The feedback loop and tool constraints prevented catastrophic action.
3. Knowledge Cut-off: The LLM's internal knowledge may not include the latest software versions or vulnerabilities, necessitating comprehensive grounding via the system prompt.

5. Discussion

The results validate the core hypothesis: LLMs can effectively serve as autonomous remediation engines for complex data pipelines. The reduction of MTTR from nearly an hour to under four minutes is transformative for healthcare operations, where data freshness can be clinically relevant.

5.1 Implications for Healthcare Data Operations

This shift from manual to autonomous remediation enhances data pipeline resilience, a key component of healthcare organization's disaster recovery and business continuity plans. It allows data engineering teams to focus on development rather than firefighting. Furthermore, the consistent application of remediation policies reduces human error and improves audit trails for compliance (HIPAA, GDPR).

5.2 Generalizability and Future Work

While tested on Kafka-Snowflake, LARF's architecture is generic and adaptable to other pipeline technologies (e.g., Amazon Kinesis to Redshift, or Google Pub/Sub to BigQuery). Future work includes:

- Fine-tuning Domain-Specific LLMs: Training or fine-tuning open-source LLMs (e.g., Llama 2) on system logs and runbooks to improve accuracy and reduce cost/latency.
- Multi-Pipeline Coordination: Enabling a single LLM agent to manage interdependent pipelines across an organization.
- Proactive Prevention: Extending the framework from remediation to prediction and prevention, using LLMs to analyze trends and suggest pre-emptive optimizations.

5.3 Ethical and Security Considerations

Granting autonomous corrective action to an AI system requires robust guardrails. The principle of least privilege must govern tool access. All actions must be logged immutably for audit. A human escalation and override channel is mandatory. The LLM must also be rigorously tested for biases that could influence its decision-making, such as consistently favoring cost-saving actions over performance-critical ones.

6. Conclusion

This research presents a novel, effective framework for applying Large Language Models to the critical challenge of operational reliability in healthcare big data pipelines. By integrating an LLM agent as a cognitive layer within the Kafka-Snowflake ecosystem, we demonstrate a transition from streaming data movement to intelligent, self-healing infrastructure. The LLM-Based Autonomous Remediation Framework (LARF) significantly reduces recovery times, adapts to complex failures, and provides human-readable reasoning for its actions. While challenges around cost, latency, and control persist, the proven efficacy of this approach marks a substantial step forward in AIOps and resilient health data engineering. As LLMs continue to evolve, their role as active governors of critical data infrastructure will become indispensable, ensuring that the lifeblood of healthcare analytics flows uninterrupted.

REFERENCES

1. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877-1901.

2. Chirumamilla, K. R. (2023). Low-Latency Data Pipelines Using Kafka and Snowflake. *JOURNAL OF RECENT TRENDS IN COMPUTER SCIENCE AND ENGINEERING (JRTCSE)*, *11*(1), 80-106.

3. Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 4-5.

4. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. *The Eastasouth Journal of Information System and Computer Science*, *2*(02), 179-188.

5. Kim, S., Kim, S., & Park, S. (2019). A real-time data stream analysis system for large-scale IoT sensor data in healthcare. Journal of Supercomputing, 75(11), 7297–7310.

6. Kleppmann, M. (2017). Designing data-intensive applications. O'Reilly Media.

7. Koteswara Rao Chirumamilla. (2023). Low-Latency Data Pipelines Using Kafka and Snowflake. *JOURNAL OF RECENT TRENDS IN COMPUTER SCIENCE AND ENGINEERING (JRTCSE)*, *11*(1), 80-106. https://doi.org/10.70589/JRTCSE.2023.1.10

8. Kreps, J. (2014). I heart logs: Event data, stream processing, and data integration. O'Reilly Media.

9. Mauro, T. (2021). Kubernetes operators: Automating the container orchestration platform. O'Reilly Media.

10. OpenAI. (2023). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.

11. Raghupathi, W., & Raghupathi, V. (2014). Big data analytics in healthcare: promise and potential. Health Information Science and Systems, 2(1), 3.

12. Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2023). Code Llama: Open foundation models for code. arXiv preprint arXiv:2308.12950.

13. Rumsfeld, J. S., Joynt, K. E., & Maddox, T. M. (2016). Big data analytics to improve cardiovascular care: promise and challenges. Nature Reviews Cardiology, 13(6), 350-359.

14. Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., ... & Varley, J. (2010). Dapper, a large-scale distributed systems tracing infrastructure. Google Research.

15. Tayal, C. (2024). Big Data Pipeline Optimisation for Electronic Health Records (EHR). *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, *5*(3), 121-127.

16. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35, 24824-24837.

17. Zhang, Y., Chen, H., Zhu, J., & Wu, D. (2020). A survey on AIOps for anomaly detection in time series. *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 366–373.

17. Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. CIDR.

18. Bates, D. W., Saria, S., Ohno-Machado, L., Shah, A., & Escobar, G. (2014). Big data in health care: using analytics to identify and manage high-risk and high-cost patients. Health Affairs, 33(7), 1123-1131.

19. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.

20. Dean, J., & Ghemawat, S. (2008). MapReduce: simplified data processing on large clusters. Communications of the ACM, 51(1), 107-113.

21. Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608.

22. Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., ... & Dean, J. (2019). A guide to deep learning in healthcare. Nature Medicine, 25(1), 24-29.

23. Gandomi, A., & Haider, M. (2015). Beyond the hype: Big data concepts, methods, and analytics. International Journal of Information Management, 35(2), 137-144.

24. Ghodsi, A., Zaharia, M., Hindman, B., Konwinski, A., Shenker, S., & Stoica, I. (2011). Dominant resource fairness: Fair allocation of multiple resource types. NsDI (Vol. 11, No. 2011, pp. 24-24).

25. Halevy, A., Norvig, P., & Pereira, F. (2009). The unreasonable effectiveness of data. IEEE Intelligent Systems, 24(2), 8-12.

26. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 770-778.

27. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.

28. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. The Eastasouth Journal of Information System and Computer Science, 2(02), 179–188. https://doi.org/10.58812/esiscs.v2i02.856

29. Kohavi, R., & Longbotham, R. (2017). Online controlled experiments and A/B testing. Encyclopedia of Machine Learning and Data Mining, 7(8), 922-929.

30. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436-444.

31. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.

32. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems, 26.

33. Murdoch, W. J., Singh, C., Kumbier, K., Abbasi-Asl, R., & Yu, B. (2019). Definitions, methods, and applications in interpretable machine learning. Proceedings of the National Academy of Sciences, 116(44), 22071-22080.

34. O'Neil, C. (2016). Weapons of math destruction: How big data increases inequality and threatens democracy. Crown.

35. Provost, F., & Fawcett, T. (2013). Data science and its relationship to big data and data-driven decision making. Big Data, 1(1), 51-59.

36. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. New England Journal of Medicine, 380(14), 1347-1358.

37. Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop Distributed File System. 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 1-10.

38. Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT press.

39. Tayal C. Big Data Pipeline Optimisation for Electronic Health Records (EHR). IJAIDSML [Internet]. 2024 Oct. 30 [cited 2026 Jan. 16];5(3):121-7. Available from: https://ijaidsml.org/index.php/ijaidsml/article/view/309

40. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

41. Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., ... & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. Presented as part of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12), 15-28.

42. Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Ghodsi, A. (2016). Apache Spark: a unified engine for big data processing. Communications of the ACM, 59(11), 56-65.

43. Zinkevich, M. (2017). Rules of machine learning: Best practices for ml engineering. Google Documentation.

44. Zobel, J., & Moffat, A. (2006). Inverted files for text search engines. ACM Computing Surveys (CSUR), 38(2), 6-es.